

AnalytiCore

Prototipo de Arquitectura Orientada a Servicios en la Nube

Documento Técnico de Arquitectura — Diagramas, Especificaciones e Informe Ejecutivo

Plataforma de despliegue: Render · Contenedorización: Docker · Persistencia: PostgreSQL

Julio 2026

Índice

1. Introducción y alcance del prototipo
2. Diagrama de Componentes
3. Diagrama de Secuencia (flujo de datos detallado)
4. Especificación de las APIs REST
5. Modelo de datos (PostgreSQL)
6. Diagramas de Capas — Arquitectura Limpia por componente
7. Diagrama de Despliegue en Render
8. Patrones de Arquitectura Cloud aplicados
9. Informe Ejecutivo
10. Conclusiones

1. Introducción y Alcance del Prototipo

Este documento describe la arquitectura técnica del prototipo AnalytiCore: una plataforma en la nube que permite a un usuario enviar un texto y recibir, de forma asíncrona, un análisis de sentimiento y una lista de palabras clave. El objetivo del prototipo es demostrar la viabilidad de una arquitectura orientada a servicios (SOA), políglota (React, Python y Java conviviendo en el mismo sistema) y desplegada íntegramente en contenedores Docker sobre la plataforma Render.

El documento cubre, en orden, el diseño de alto nivel (diagrama de componentes), el comportamiento en el tiempo (diagrama de secuencia), el contrato de las APIs expuestas, el modelo de datos persistido en PostgreSQL, el diseño interno de cada componente bajo el enfoque de Arquitectura Limpia (Clean Architecture), la estrategia de despliegue en la nube, los patrones de arquitectura cloud exigidos y, finalmente, el informe ejecutivo dirigido a una audiencia no técnica.

Alcance funcional

- Enviar un texto para análisis desde una interfaz web.
- Procesar el texto de forma asíncrona: análisis de sentimiento (positivo / negativo / neutral) y extracción de palabras clave.
- Consultar en cualquier momento el estado de un trabajo (PENDIENTE, PROCESANDO, COMPLETADO) y sus resultados.

Fuera de alcance (para futuras iteraciones)

- Autenticación y autorización de usuarios.
- Modelos de análisis de sentimiento avanzados (NLP profundo / IA generativa).
- Notificaciones push o WebSockets (en este prototipo el estado se obtiene por polling).

2. Diagrama de Componentes

El siguiente diagrama muestra los tres componentes principales del sistema —cada uno empaquetado como un contenedor Docker independiente—, la base de datos PostgreSQL gestionada por Render, y el flujo de comunicación entre ellos, exclusivamente a través de APIs REST.

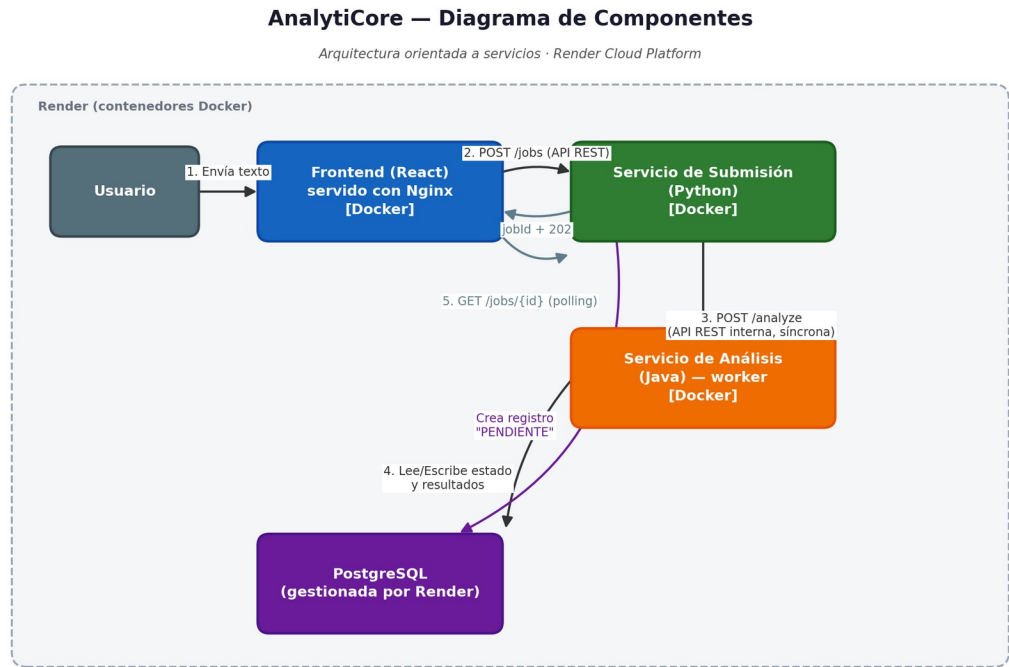


Figura 1. Diagrama de componentes de AnalytiCore, incluyendo el flujo de datos numerado (pasos 1 a 5).

Responsabilidad de cada componente

Componente	Tecnología	Responsabilidad principal
Frontend	React + Nginx	Captura el texto del usuario, envía la solicitud, y hace polling del resultado mostrando el estado en pantalla.
Servicio de Submisión	Python (FastAPI)	Valida la entrada, crea el registro inicial del trabajo, orquesta la llamada al servicio de análisis y expone el estado al frontend.
Servicio de Análisis	Java (Spring Boot)	Ejecuta el procesamiento real del texto (sentimiento + palabras clave) y actualiza el resultado en la base de datos.
Base de datos	PostgreSQL (managed)	Única fuente de verdad del estado de negocio; almacena cada trabajo y su resultado.

3. Diagrama de Secuencia — Flujo de Datos Detallado

Mientras el diagrama de componentes muestra la estructura estática del sistema, el diagrama de secuencia muestra el orden temporal exacto de los mensajes intercambiados entre el usuario, los tres servicios y la base de datos para completar un ciclo de análisis.

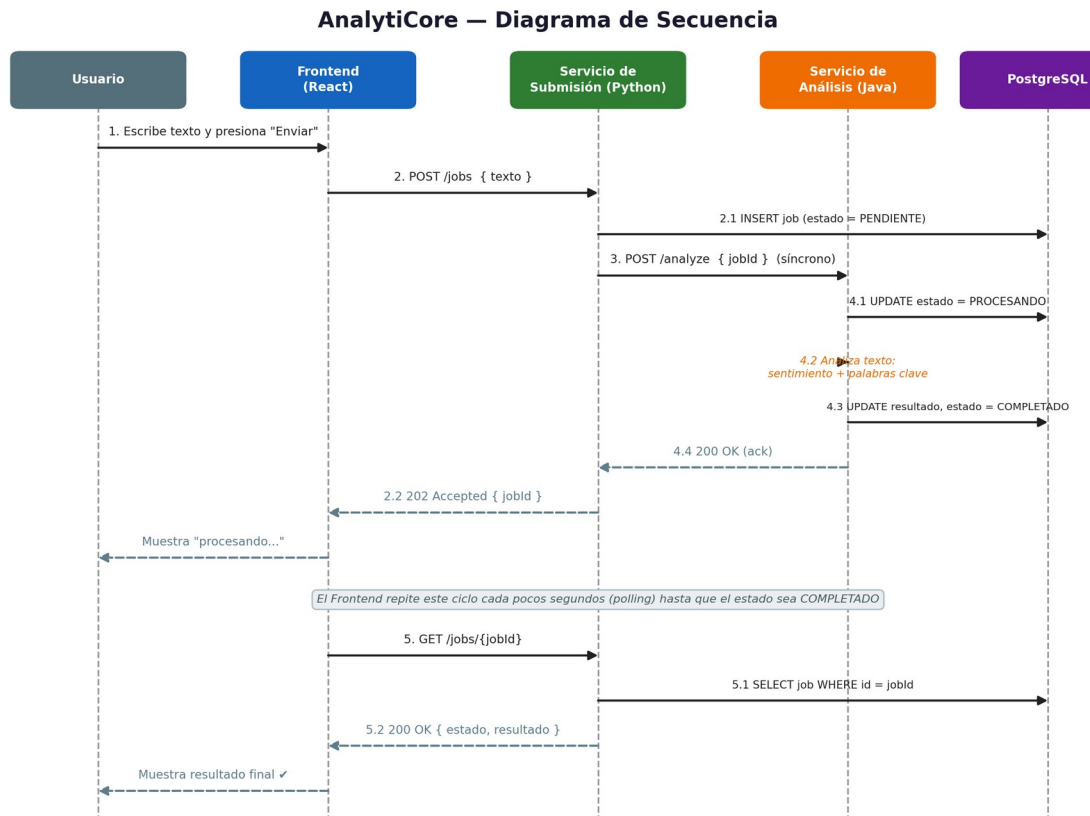


Figura 2. Diagrama de secuencia del flujo completo de un trabajo de análisis, desde el envío hasta la consulta del resultado.

Puntos clave a resaltar de este diagrama:

- El paso 3 (Servicio de Submisión → Servicio de Análisis) es una llamada **SÍNCRONA**: el servicio Python espera la confirmación de que el trabajo fue recibido, pero no espera a que termine el análisis completo — por eso el frontend recibe rápidamente su jobId y comienza a hacer polling.
- El paso 4 ocurre en segundo plano dentro del Servicio de Análisis, sin bloquear al usuario.
- El polling (pasos 5 en adelante) se repite cada pocos segundos hasta que el estado almacenado en PostgreSQL es COMPLETADO.

4. Especificación de las APIs REST

Toda la comunicación entre componentes se realiza mediante APIs RESTful que intercambian JSON. A continuación se documenta el contrato de cada endpoint, siguiendo un formato similar a OpenAPI/Swagger.

4.1 API pública — Servicio de Submisión (Python)

Método	Ruta	Descripción	Cuerpo / Respuesta
POST	/jobs	Crea un nuevo trabajo de análisis a partir de un texto.	Body: { "text": string } Resp 202: { "jobId": string, "status": "PENDIENTE" }
GET	/jobs/{jobId}	Consulta el estado y resultado de un trabajo existente.	Resp 200: { "jobId", "status", "sentiment"?, "keywords"? }

4.2 API interna — Servicio de Análisis (Java)

Método	Ruta	Descripción	Cuerpo / Respuesta
POST	/analyze	Recibida solo por el Servicio de Submisión. Dispara el procesamiento del texto para un jobId.	Body: { "jobId": string, "text": string } Resp 200: { "jobId", "status": "PROCESANDO" }

Nota: el endpoint /analyze no debe exponerse públicamente en producción; en Render se recomienda restringirlo a red interna o protegerlo con una clave compartida entre servicios (API key interna).

5. Modelo de Datos (PostgreSQL)

Todo el estado del negocio se externaliza a una única tabla principal, jobs, que centraliza el ciclo de vida completo de un trabajo de análisis. Esto es lo que permite que los servicios Python y Java sean stateless: ninguno guarda el progreso en su propia memoria, todos leen y escriben en esta tabla.

Columna	Tipo	Descripción
id	UUID (PK)	Identificador único del trabajo (el jobId devuelto al frontend).
input_text	TEXT	Texto original enviado por el usuario.
status	VARCHAR	PENDIENTE PROCESANDO COMPLETADO ERROR.
sentiment	VARCHAR (nullable)	Resultado del análisis: positivo / negativo / neutral.
keywords	TEXT[] / JSONB (nullable)	Lista de palabras clave extraídas del texto.
created_at	TIMESTAMP	Fecha y hora de creación del trabajo.
updated_at	TIMESTAMP	Fecha y hora de la última actualización de estado.

Diseñar el estado como una máquina de estados simple (PENDIENTE → PROCESANDO → COMPLETADO, con ERROR como camino alterno) facilita que cualquier instancia de cualquier servicio pueda atender una solicitud sin depender de qué instancia procesó el paso anterior — un requisito directo del patrón "servicios sin estado".

6. Diagramas de Capas (Arquitectura Limpia)

Cada uno de los tres componentes se diseña siguiendo los principios de Arquitectura Limpia (Clean Architecture) de Robert C. Martin: las capas externas dependen de las internas y nunca al revés, de modo que las reglas de negocio (el dominio) no dependen de frameworks, bases de datos ni interfaces de usuario concretas. Esto se representa como círculos concéntricos: cuanto más al centro, más estable y más independiente de la tecnología es el código.

6.1 Frontend (React)

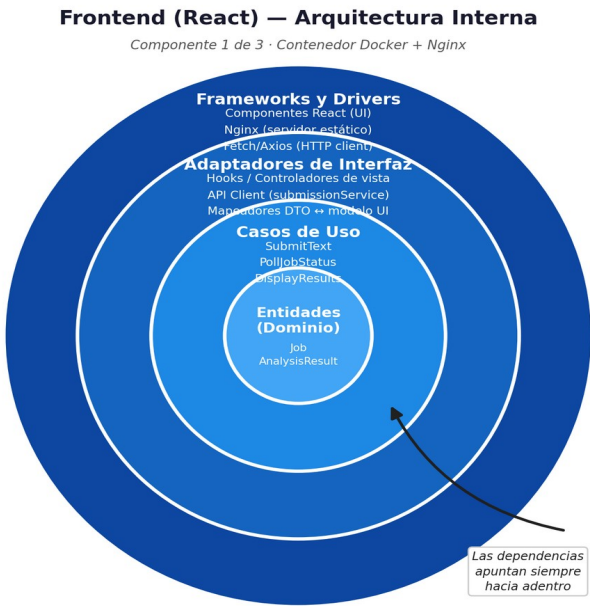


Figura 3. Arquitectura interna del Frontend.

Capa	Contenido	Por qué está ahí
Entidades (Dominio)	Job, AnalysisResult	Representan los conceptos del negocio, sin ninguna dependencia de React ni de HTTP.
Casos de uso	SubmitText, PollJobStatus, DisplayResults	Orquestan la lógica de la aplicación: qué pasos seguir, en qué orden.
Adaptadores	Hooks, cliente API (submissionService), mapeadores DTO	Traducen entre el mundo externo (HTTP/JSON) y las entidades internas.
Frameworks/Drivers	Componentes React, Nginx	Detalles de implementación reemplazables: hoy es React, podría ser otro framework sin tocar el dominio.

6.2 Servicio de Submisión (Python)

Servicio de Submisión (Python) — Arquitectura Interna

Componente 2 de 3 · Contenedor Docker

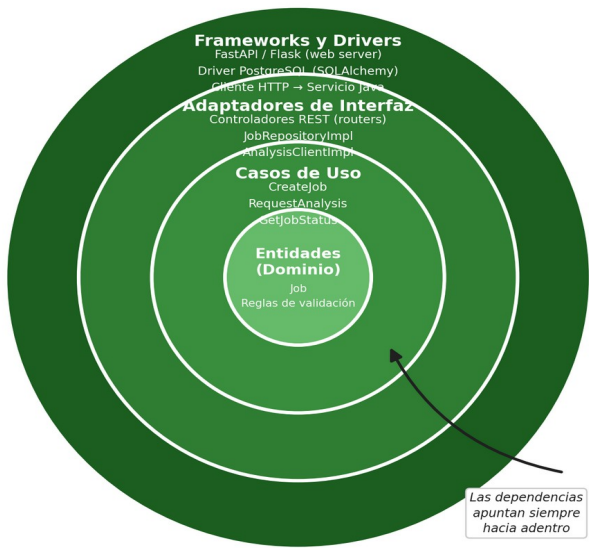


Figura 4. Arquitectura interna del Servicio de Submisión.

Capa	Contenido	Por qué está ahí
Entidades (Dominio)	Job, reglas de validación	Definen qué es un trabajo válido, independientemente de FastAPI o de PostgreSQL.
Casos de uso	CreateJob, RequestAnalysis, GetJobStatus	Contienen la lógica de aplicación: crear, orquestar y consultar.
Adaptadores	Routers/controladores REST, JobRepositoryImpl, AnalysisClientImpl	Exponen la API HTTP y traducen las llamadas a los casos de uso.
Infraestructura	SQLAlchemy (PostgreSQL), cliente HTTP al servicio Java	Detalles técnicos concretos, aislados detrás de interfaces.

6.3 Servicio de Análisis (Java)

Servicio de Análisis (Java) — Arquitectura Interna
Componente 3 de 3 · Contenedor Docker · Worker

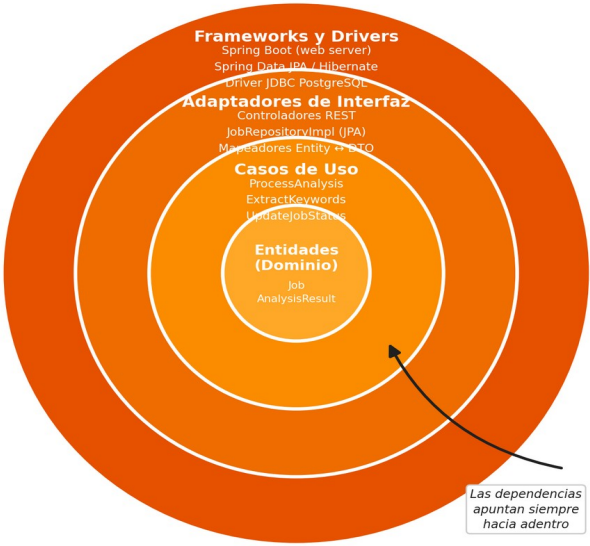


Figura 5. Arquitectura interna del Servicio de Análisis.

Capa	Contenido	Por qué está ahí
Entidades (Dominio)	Job, AnalysisResult	El núcleo de negocio del análisis, sin dependencias de Spring.
Casos de uso	ProcessAnalysis, ExtractKeywords, UpdateJobStatus	Definen el algoritmo de negocio del análisis de texto.
Adaptadores	Controladores REST, repositorios JPA	Exponen /analyze y traducen entidades a registros de base de datos.
Infraestructura	Spring Boot, Spring Data JPA/Hibernate, driver JDBC	El framework y el motor de persistencia concretos.

7. Diagrama de Despliegue en Render

En producción, cada uno de los tres componentes se despliega como un Web Service independiente en Render, construido a partir de su Dockerfile. La base de datos PostgreSQL se aprovisiona como un servicio gestionado (managed database) separado, y todos los servicios se conectan a ella mediante una cadena de conexión inyectada por variable de entorno.

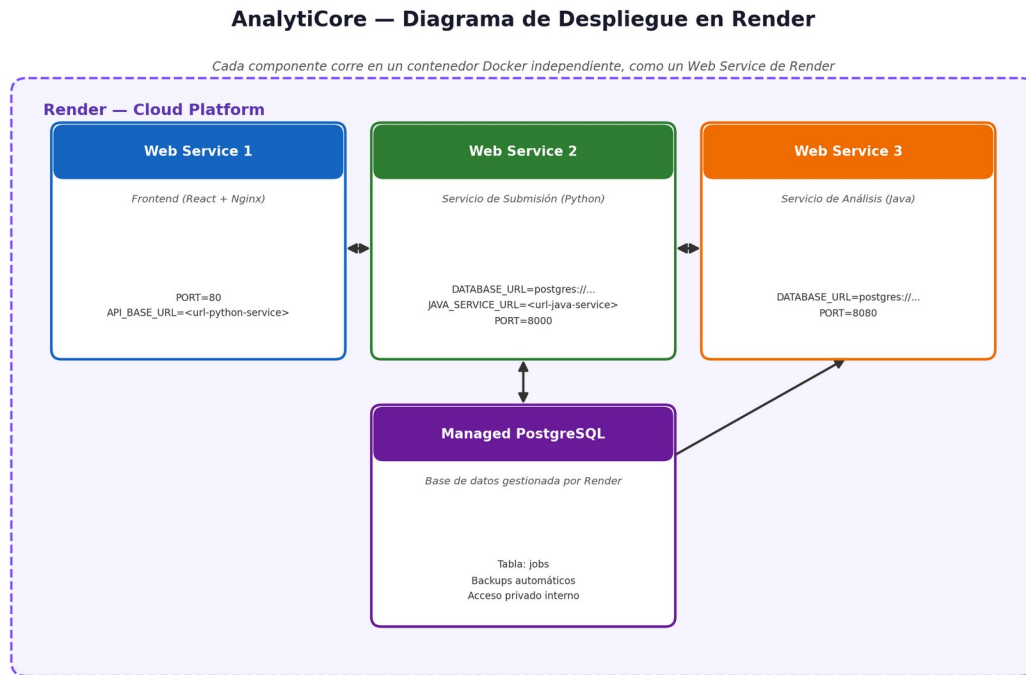


Figura 6. Diagrama de despliegue de AnalytiCore sobre Render.

Cada Web Service se configura de forma completamente independiente: tiene su propio proceso de build (a partir del Dockerfile de su carpeta), su propio ciclo de vida, su propio auto-escalado y sus propias variables de entorno. Ninguno conoce detalles internos de los otros: solo conocen su URL pública y, en el caso de Python y Java, la cadena de conexión a PostgreSQL.

8. Patrones de Arquitectura Cloud Aplicados

A continuación se detalla cómo el prototipo satisface cada uno de los cinco patrones de arquitectura cloud exigidos por el enunciado del proyecto.

8.1 Empaquetado de las aplicaciones en contenedores

Cada componente (Frontend, Python, Java) se define en su propio Dockerfile, ubicado en la raíz de su carpeta, y se construye como una imagen independiente configurada para producción: sin herramientas de desarrollo, sin recarga en caliente, y usando builds multi-stage donde aplica (por ejemplo, compilar el bundle de React y luego servirlo con una imagen liviana de Nginx, o compilar el .jar de Java y ejecutarlo sobre una imagen JRE mínima).

8.2 Exposición de componentes como APIs

No existe ninguna comunicación directa entre componentes que no sea a través de HTTP/REST con JSON. El Frontend nunca accede directamente a PostgreSQL ni al Servicio de Análisis: siempre pasa por la API del Servicio de Submisión. Esto permite que cada componente evolucione, se reemplace o se escale de forma independiente sin romper a los demás.

8.3 Servicios sin estado (stateless)

Ni el Servicio de Submisión ni el Servicio de Análisis mantienen variables en memoria que representen el progreso de un trabajo. Todo el estado (PENDIENTE, PROCESANDO, COMPLETADO, y los resultados) vive exclusivamente en PostgreSQL. Esto significa que si Render decide reiniciar un contenedor, o si en el futuro se ejecutan varias instancias del mismo servicio para escalar, cualquier instancia puede atender cualquier solicitud sin perder información.

8.4 Configuración externa de PostgreSQL

Ningún servicio tiene la cadena de conexión, el usuario o la contraseña de la base de datos escritos en el código fuente. Todos se leen desde variables de entorno (DATABASE_URL y similares), definidas en un archivo .env en desarrollo local y en la sección de variables de entorno de cada Web Service dentro del panel de Render en producción.

8.5 Arquitectura limpia en los tres componentes

Como se detalla en la sección 6, cada componente separa explícitamente sus reglas de negocio (dominio y casos de uso) de los detalles técnicos (frameworks, bases de datos, HTTP). Esto reduce el acoplamiento, facilita las pruebas unitarias del dominio sin necesidad de levantar una base de datos real, y permite que el equipo cambie una tecnología concreta (por ejemplo, migrar de SQLAlchemy a otro ORM) sin tener que rediseñar la lógica de negocio.

9. Informe Ejecutivo

(Extensión máxima de una página, dirigido a una audiencia no técnica.)

Problema de negocio

AnalytiCore es una startup de análisis de datos que necesita ofrecer, de forma rápida y confiable, un servicio en línea donde cualquier usuario pueda enviar un texto y recibir un análisis de sentimiento junto con las palabras clave más relevantes. Hoy no existe una plataforma propia que resuelva esto: se requiere un producto que funcione en la nube, que pueda crecer conforme aumenten los usuarios, y que sirva como base sólida para nuevas funcionalidades en el futuro.

Solución propuesta y valor agregado

La solución construida es una arquitectura orientada a servicios (SOA) desplegada en Render, compuesta por tres aplicaciones independientes, cada una en su propio contenedor Docker: un Frontend en React que ofrece la interfaz al usuario, un Servicio de Submisión en Python que recibe y valida las solicitudes, y un Servicio de Análisis en Java que procesa el texto y genera los resultados. Los tres se comunican exclusivamente mediante APIs REST, y toda la información del negocio se guarda de forma centralizada en una base de datos PostgreSQL administrada por Render.

El valor de esta solución es que entrega un prototipo funcional y realista de cómo operaría el producto en producción: demuestra que equipos distintos pueden trabajar cada uno en su propia tecnología sin bloquearse entre sí, y deja una base técnica lista para agregar nuevas funciones sin tener que rediseñar el sistema desde cero.

Beneficios de la arquitectura elegida

- Escalabilidad: al ser servicios independientes y sin estado, cada componente puede crecer por separado añadiendo más instancias solo donde se necesite.
- Mantenibilidad: la arquitectura limpia facilita corregir errores o modificar una parte del sistema sin arriesgar a romper las demás.
- Flexibilidad del equipo: distintos desarrolladores pueden especializarse en Python, Java o React y trabajar en paralelo, o incluso reemplazar un componente completo sin afectar al resto.
- Confiabilidad de los datos: al externalizar todo el estado a PostgreSQL, ningún trabajo se pierde si un contenedor se reinicia.

10. Conclusiones

El prototipo AnalytiCore demuestra, con un caso de uso simple pero completo, que es posible construir un sistema políglota, desacoplado y desplegable en la nube siguiendo buenas prácticas de arquitectura de software: contenedorización, comunicación exclusiva por APIs REST, servicios sin estado, configuración externa y arquitectura limpia en cada componente.

Esta base es extensible: agregar un nuevo tipo de análisis, un nuevo canal de entrada (por ejemplo, una API pública para integraciones), o incluso un cuarto microservicio, no debería requerir modificar la lógica interna de los componentes existentes, sino únicamente añadir un nuevo adaptador o un nuevo caso de uso en el componente correspondiente. Esa es, precisamente, la promesa central de la arquitectura orientada a servicios: que el sistema pueda crecer sin que cada cambio se convierta en un riesgo para el resto de la plataforma.